CLOUD & DEVOPS ACADEMY

Cloud & DevOps Engineering

The Engineering Programme

A sixteen-week professional syllabus. Infrastructure, delivery, and operations, taught as one system and proved in a capstone.

THE PATH

Linux · Git · GitLab CI
AWS · Terraform · Ansible · Docker
Kubernetes · Argo CD
Prometheus · Alertmanager · Grafana · ELK

2026 SYLLABUS PRACTICAL STANDARD CAPSTONE ASSESSED

The programme

Graduates leave with a platform they can defend: infrastructure in code, a delivery pipeline, a Kubernetes release path, and the signals required to operate it. The work is hands-on from the first week.

16	3	70%	1
Weeks, one continuous build	Instructor-led sessions each week	Labs, projects, and review	Capstone, demonstrated live

WHO IT IS FOR

Beginners with sound computer skills, IT students, developers, and engineers moving into cloud and DevOps. Basic Linux is useful. Learners who need it receive a short orientation before Week 1. No prior cloud account is required.

BY THE END, A GRADUATE CAN

01 Administer Linux and troubleshoot networking, access, and services.

02 Collaborate in Git and GitLab, and ship work through merge requests.

03 Build, run, and harden Docker images and Compose applications.

04 Provision AWS with least privilege and express it in Terraform.

05 Configure systems with Ansible and keep runs idempotent.

06 Design GitLab CI pipelines that test, scan, publish, and roll back.

07 Deploy and operate applications on Kubernetes, including GitOps with Argo CD.

08 Monitor with Prometheus, Alertmanager, and Grafana, and investigate with ELK.

Sixteen weeks

Tools are taught in the order a system is built. The machine and the repository come first. Then the cloud account, the container, the pipeline, the cluster, and the signals that show whether it is healthy.

MONTH 01

Foundations

Linux, the network, Git, and a first AWS account used with care.

- 01 Linux and the shell
- 02 Networking and SSH
- 03 Git and GitLab
- 04 AWS fundamentals

MONTH 02

Provision

Containers, a resilient AWS design, Terraform, and Ansible.

- 05 Docker
- 06 AWS architecture
- 07 Terraform
- 08 Ansible

MONTH 03

Delivery

GitLab CI, Kubernetes, Helm, and security inside the pipeline.

- 09 GitLab CI
- 10 Kubernetes
- 11 Cluster operations
- 12 DevSecOps

MONTH 04

Operations

Prometheus, Alertmanager, Grafana, ELK, Argo CD, and the capstone.

- 13 Metrics and alerts
- 14 Logging with ELK
- 15 Argo CD
- 16 Capstone

Each week ends with evidence: a repository, a working system, and a short account of what was decided. The capstone assembles those pieces into one platform.

WEEKS 01 • 04

Foundations

Linux to AWS

01 Linux administration and the shell

Distributions, the filesystem, and navigation. Users, groups, permissions, ownership, and sudo. Packages, processes, systemd, logs, and disk. Bash variables, pipes, redirection, grep, find, awk, sed, and scripts.

EVIDENCE A documented server baseline with least-privilege accounts, and a Bash report of disk, memory, uptime, and service health.

02 Networking, SSH, and troubleshooting

IP addressing, CIDR, DNS, DHCP, routing, ports, and common protocols. HTTP, TLS, firewalls, and security groups. SSH keys, scp, rsync, curl, ping, traceroute, dig, and ss. Hardening, patching, and secrets hygiene.

EVIDENCE Two Linux machines with key-based SSH, and a runbook for a deliberately broken web service.

03 Git and GitLab collaboration

Repositories, commits, branches, merges, tags, reset, revert, and conflicts. GitLab issues, merge requests, review, and protected branches. READMEs, branching policy, semantic versioning, and release notes. YAML as the team•s configuration language.

EVIDENCE A team project with feature branches, reviews, issues, and tagged releases, plus a small application and its release notes.

04 Cloud concepts and AWS fundamentals

Service and deployment models, regions, Availability Zones, and shared responsibility. IAM users, roles, policies, MFA, least privilege, and the AWS CLI. EC2, EBS, S3, VPC, subnets, route tables, gateways, and security groups. Budgets, tagging, CloudTrail, and lab cleanup.

EVIDENCE A web server on EC2 inside a restricted VPC, and a static site on S3 with the security and cost decisions written down.

WEEKS 05 · 08

Provision

Image to code

05 Docker and containerization

Containers and virtual machines. Images, registries, layers, and the Docker architecture. Dockerfiles, build context, tags, and multi-stage builds. Ports, volumes, networks, environment, and health checks. Compose, logs, resource limits, and image scanning.

EVIDENCE An application and database in Compose, with an optimized multi-stage image, persistent volumes, and notes on how to run and troubleshoot it.

06 AWS architecture

Launch templates, Auto Scaling, and an Application Load Balancer. S3 lifecycle and versioning, IAM roles, Lambda, and event-driven patterns. CloudWatch metrics, logs, and alarms. Public and private subnets, NAT, Secrets Manager, and Systems Manager.

EVIDENCE A highly available sample behind a load balancer with health checks and least-privilege roles, plus a Lambda with logs and a CloudWatch alarm.

07 Terraform

Providers, resources, data sources, variables, outputs, and locals. Init, validate, plan, apply, destroy, and state. Modules, remote state, locking, workspaces, and environment separation. Secrets, plan review, drift, tagging, and cost control.

EVIDENCE A VPC, subnets, security groups, an EC2 instance, and an S3 bucket in Terraform, refactored into modules for dev and test, then destroyed cleanly.

08 Ansible

Inventory, ad hoc commands, modules, facts, and playbooks. Variables, handlers, templates, roles, loops, conditionals, and Ansible Vault. Idempotency, SSH, and privilege escalation. Where Terraform provisions and Ansible configures.

EVIDENCE A role that installs Nginx, deploys a page, and sets a firewall rule, then a Terraform-built host configured by Ansible with a repeat run that changes nothing.

WEEKS 09 · 12

Delivery

Pipeline to cluster

09 CI/CD with GitLab

Stages, jobs, runners, artifacts, caches, and variables. GitLab CI YAML, rules, needs, environments, and manual approvals. Build, test, package, and publish images to a registry. Rolling, blue/green, and canary releases, plus rollback and release gates.

EVIDENCE A pipeline that lints, tests, builds, and pushes an image on merge to the default branch, then a test deploy and a demonstrated rollback.

10 Kubernetes core

Control plane, nodes, API server, scheduler, and etcd. Pods, Deployments, ReplicaSets, Services, namespaces, labels, and selectors. ConfigMaps, Secrets, probes, requests, limits, and rolling updates. kubectl and a local cluster with kind or Minikube.

EVIDENCE A multi-replica application with a Service, a ConfigMap, and probes, then a broken rollout diagnosed from events, logs, and history.

11 Kubernetes networking, storage, and operations

Ingress, service discovery, and network policy. PersistentVolumes, claims, StorageClasses, and stateful workloads. The Horizontal Pod Autoscaler, scheduling, and availability. Helm values, upgrades, and rollback. Security contexts and RBAC.

EVIDENCE A Helm chart with configurable replicas, resources, and environment, plus a stateful workload and a restricted service account.

12 DevSecOps in the pipeline

Secure delivery, a light threat model, and checks that run in GitLab CI. Secret scanning, dependency scanning, SAST and DAST concepts, and a software bill of materials. Minimal images, non-root containers, and vulnerability scans. Terraform policy checks, provenance, and approvals.

EVIDENCE Code, dependency, container, and infrastructure checks in the pipeline, with findings fixed, and a hardened image: non-root, no embedded secrets, versions pinned.

WEEKS 13 · 16

Operations

Signal to release

13 Prometheus, Grafana, and Alertmanager

Metrics, logs, and traces. Golden signals, SLIs, SLOs, and alert design that stays quiet. Prometheus scrape, exporters, labels, PromQL, and alert rules. Alertmanager grouping, routing, inhibition, silences, and receivers. Grafana dashboards, variables, and the difference between a chart and an alert path.

EVIDENCE An instrumented application and host, a Grafana dashboard, and Alertmanager routes for high CPU, low disk, and downtime, with a tested response note.

14 Logging with the ELK stack

Structured logs, levels, correlation IDs, and fields worth keeping. Elasticsearch indexes and mappings. Kibana Discover, visualizations, and dashboards. Ingestion with Elastic Agent, Filebeat, or Logstash. Retention, access control, redaction, and how a log investigation actually proceeds.

EVIDENCE Application and system logs in Elasticsearch, a Kibana dashboard, and a written investigation that names an error and its cause.

15 GitOps with Argo CD

Desired state, reconciliation, and declarative delivery. Argo CD applications, repositories, sync policy, health, and drift. Application source kept apart from deployment manifests. Promotion across environments, a secrets approach, progressive delivery, approvals, rollback, and runbooks.

EVIDENCE A Kubernetes release deployed from Git, a drift correction, a promotion from staging to a production-like namespace, and a rollback.

16 Capstone, review, and demonstration

Architecture, reliability, security, cost, and documentation judged together. An end-to-end pass over the pipeline, an incident exercise, and a handover. Portfolio, presentation, and a technical review.

EVIDENCE The platform described on the next page, demonstrated live, with the repository, diagrams, dashboards, and runbook submitted.

WEEK 16 · FINAL EVIDENCE

The capstone

Design, automate, deploy, secure, and observe a production-style application. A task manager, an inventory service, a student portal, or an API. Work is individual or in a small team. The standard is a system another engineer could operate from the runbook.

01	**Source**	GitLab, with a documented branching model and merge-request workflow.
02	**Infrastructure**	Terraform on AWS. Least privilege, tags, and a short cost note. Ansible where a host must be configured.
03	**Pipeline**	GitLab CI that validates, tests, builds an image, scans it, and pushes it to a registry.
04	**Cluster**	Kubernetes through Helm or manifests. Probes, requests and limits, and RBAC. EKS where budget allows; kind or Minikube otherwise.
05	**Release**	Argo CD sync from Git, with evidence of promotion and rollback.
06	**Metrics**	Prometheus, Grafana, and Alertmanager. A dashboard and alert routes that have been fired and answered.
07	**Logs**	Centralized logs in the ELK stack and a Kibana view used for an investigation.
08	**Record**	Architecture diagram, README, security findings and fixes, operations runbook, and a live demonstration.

WHAT THE REVIEW ASKS

Explain the security decisions, how the system recovers from a bad release, which signals would page someone, and how cost is kept in bounds. The diagram and the running system should describe the same architecture.

BUDGET DISCIPLINE

EKS and always-on cloud labs are optional. Core Kubernetes work runs locally. AWS is used for selected integration labs and is torn down when the work is done. Cost control is part of the grade, not an afterthought.

Assessment

Grades follow finished work. A claim without a repository, a diagram, or a demonstration does not count.

ASSESSMENT	WEIGHT	WHAT IS JUDGED
Weekly practical labs	30%	Completion, troubleshooting, and evidence from the week's exercises.
Topic projects and reviews	20%	Working deliverables, documentation, and the hygiene of the Git history.
Mid-programme practical	15%	Docker, AWS, Terraform, Ansible, and a working CI/CD path.
Security and observability	10%	Pipeline checks, dashboards, logs, and alerts that fire for a real reason.
Final capstone	25%	Architecture, automation, security, reliability, cost, and the live demonstration.

THE SHAPE OF A WEEK

SESSION 1
Concepts

The idea, a worked demonstration, and the decisions the lab will force.

SESSION 2
Guided lab

Instructor-led practice with checkpoints where things are expected to break.

SESSION 3
Workshop

Project work, review of someone else's change, a short check, and a presentation.

BETWEEN SESSIONS
Independent

Two to four hours for practice, documentation, and the portfolio.

TO COMPLETE THE PROGRAMME

- Finish at least 80 percent of the labs and weekly assignments.
- Submit the topic projects and pass the mid-programme practical.
- Demonstrate the capstone and submit source, infrastructure, and the runbook.
- Explain security, recovery, monitoring, and cost in the final review.

Studio standard

The lab is part of the curriculum. Accounts, machines, and secrets are handled the way a professional team handles them.

PRACTICE

01 A laptop with 8 GB of memory at minimum. 16 GB is the recommendation once local Kubernetes begins.

02 Linux via a virtual machine, WSL2, or a dedicated machine. GitLab access and a container registry are provided.

03 AWS learner accounts use MFA, budget alerts, least privilege, and a written cleanup at the end of every lab.

04 Kubernetes exercises run on kind or Minikube. EKS is reserved for integration work when the budget allows.

05 Credentials never go into Git. Labs use temporary credentials and an approved secrets mechanism.

06 Every project leaves a repository, a README, a diagram where the system has one, and a short demonstration.

TOOL STACK

SYSTEMS

Linux, Bash, SSH, and the usual network tools.

COLLABORATION

Git and GitLab, including issues and merge requests.

CLOUD

IAM, VPC, EC2, S3, Lambda, CloudWatch, load balancing, and EKS as a concept.

AUTOMATION

Terraform and Ansible.

DELIVERY

Docker, Compose, GitLab CI, a registry, Kubernetes, Helm, kind or Minikube, and Argo CD.

OBSERVABILITY

Prometheus, Alertmanager, Grafana, and the ELK stack.

SECURITY CHECKS

Secret, dependency, image, and infrastructure scanning in the pipeline.